COSC3506/ITEC3506 Software Engineering

Module 6 Lesson 1

Coupling and Cohesion

Dr. Simon Xu

1

Outline

- Modules
- Cohesion
- Coupling

2

Modules

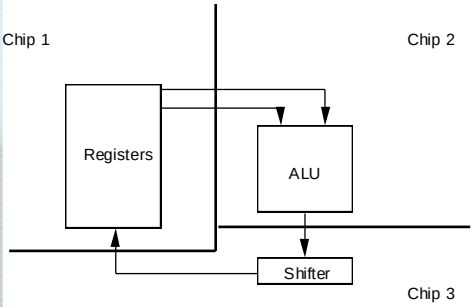
- **What is a module?**
  - **lexically contiguous sequence of program statements, bounded by boundary elements, with aggregate identifier**
- **Examples**

3

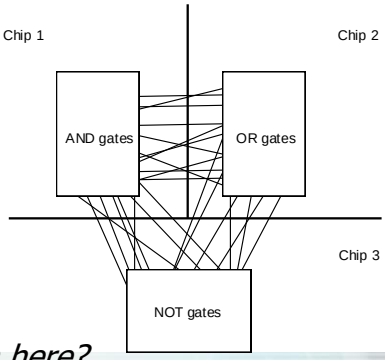
Good vs. Bad Modules

- **Modules in themselves are not "good"**
- **Must design them to have good properties**

i) CPU using 3 chips (modules)



ii) CPU using 3 chips (modules)



What is the problem here?

4

Cohesion ?

- Degree of interaction within module
- Seven levels of cohesion
  - 7. Functional | Informational (best)
  - 5. Communicational
  - 4. Procedural
  - 3. Temporal
  - 2. Logical
  - 1. Coincidental (worst)

5

1. Coincidental Cohesion

- Def. ?
  - module performs multiple, completely unrelated actions
- How could this happen?
  - hard organizational rules about module size
- Why is this bad?
  - degrades maintainability & modules are not reusable
- Easy to fix. How?
  - break into separate modules each performing one task

6

2. Logical Cohesion

- Def.
  - module performs series of related actions, one of which is selected by calling module
- Example 1
- Why is this bad?
  - interface difficult to understand
  - code for more than one action may be intertwined
  - difficult to reuse
- Example 2: an object that performs all input and output

7

3. Temporal Cohesion

- Def. ?
  - module performs series of actions related in time (within the same limit time period)
- Why is this bad?
  - actions weakly related to one another, but strongly related to actions in other modules
  - code spread out -> not maintainable or reusable

8

4. Procedural Cohesion

- **Def. ?**
  - **module performs series of actions related by procedure to be followed by product**
- **Example**
  - **Read part number from database and update repair record on maintenance file**
- **Why is this bad?**
  - **actions are still weakly related to one another**
  - **not reusable**

9

5. Communicational Cohesion

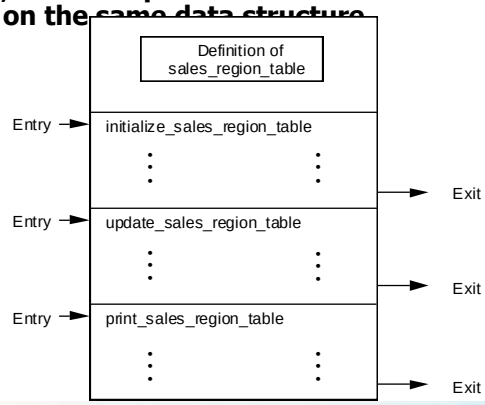
- **Def.**
  - **module performs series of actions related by procedure to be followed by product, but in addition all the actions operate on same data**
- **Example**
  - **Calculate new coordinators and send them to window**
- **Why is this bad?**
  - **still leads to less reusability -> break it up**

10

7. Informational Cohesion

- **Def.**
  - **module performs a number of actions, each with its own entry point, with independent code for each action, all performed on the same data structure**

This is an ADT!



11

7. Functional Cohesion

- **Def.**
  - **module performs exactly one action or achieve a single goal**
- **Why is this good?**
  - **more reusable**
  - **corrective maintenance easier**
    - **fault isolation**
    - **reduced regression faults**
  - **easier to extend product**
    - **Hard drive example**

12

Cohesion ?

- **Degree of interaction within module**
- **Example: Compute average daily temperature at various sites**

13

Coupling ?

- **Degree of interaction between two modules**
- **Five levels of coupling**
  - **5. Data** **(best)**
  - **4. Stamp**
  - **3. Control**
  - **2. Common**
  - **1. Content** **(worst)**

14

1. Content Coupling

- **Def.**
  - **one module directly references contents of the other**
- **Example**
  - **module `a` modifies statements of module `b`**
  - **module `a` refers to local data of module `b` in terms of some numerical displacement within `b`**
- **Why is this bad?**
  - **almost any change to `b` requires changes to `a`**

15

2. Common Coupling

- **Def.**
  - **two modules have write access to the same global data**
- **Why is this bad?**
  - **resulting code is unreadable**
    - **modules can have side-effects**
    - **must read entire module to understand**
  - **difficult to reuse**
  - **module exposed to more data than necessary**

16

3. Control Coupling

- Def.
  - one module passes an element of control to the other
- Example
  - control-switch passed as an argument
- Why is this bad?
  - modules are not independent
    - module `b` must know the internal structure of module `a`
    - affects reusability

17

4. Stamp Coupling

- Def.
  - data structure is passed as parameter, but called module operates on only some of individual components
- Why is this bad?
  - affects understanding
    - not clear, without reading entire module, which fields of record are accessed or changed
  - unlikely to be reusable
    - other products have to use the same higher level data structures
  - passes more data than necessary
    - e.g., uncontrolled data access can lead to computer crime

18

5. Data Coupling

- **Def.**
  - **every argument is either a simple argument or a data structure in which all elements are used by the called module**
- **Why is this good?**
  - **maintenance is easier**
  - **good design has high cohesion & weak coupling**

19

Abstract Data Types

- **Def.**
  - **data type together with actions to be performed on instantiations of that data type**
- **Example**

```
class JobQueue {
  public int     queueLength;     // length of job queue
  public int     queue =  new int[25];     // up to 25 jobs
  // methods
  public void initializeJobQueue (){
        // body of method
  }
  public void addJobToQueue (int jobNumber){
        // body of method
  }
} // JobQueue
```


20

Information Hiding

- **Really "details hiding"**
  - **hiding implementation details, not information**
- **Example**
  - **design modules so that items likely to change are hidden**
    - **future change localized**
    - **change cannot affect other modules**
  - **data abstraction**
    - **designer thinks at level of ADT**

21

Information hiding

- **Example**

```
class JobQueue {
  private int    queueLength;     // length of job queue
  private int    queue =  new int[25];     // up to 25 jobs
  // methods
  public void initializeJobQueue (){
        // body of method
  }
  public void addJobToQueue (int jobNumber){
        // body of method
  }
} // JobQueue
```

  - **Now `queue` and `queueLength` are inaccessible**

22

Summary

- **Good modules have strong cohesion and weak coupling**
- **Data encapsulation, ADTs, information hiding, & objects ease maintenance & reuse**

23